

“Semaphore” 之一



学会使用Semaphore

北京理工大学计算机学院
金旭亮

概述



Semaphore主要的作用是控制线程并发的数量，被称为“**信号量**”。



Semaphore管理着若干张“运行许可证 (Permit)”，它从“**申请 (acquire)**”这一许可证的多个线程中按照特定策略选择一个线程，颁发给它许可证。



得到许可证的线程可以投入运行，当此线程运行结束，将许可证返给给Semaphore，这个操作称为“**释放 (Release)**”，以便给其它的线程以运行的机会。



acquire与release必须配对。

“信号量”的含义与作用



一个信号量就是一个计数器，可用于保护对一个或多个共享资源的访问。

当一个线程要访问多个共享资源中的一个时，JVM检查信号量的计数值。如果信号量计数器当前值大于0，那么信号量就递减计数器值并允许线程访问。计数器的值大于0意味着存在可用的空闲资源，所以线程能够访问并使用这些资源中的一个。

如果计数器的值为0，JVM会让线程休眠，直到计数器的值大于0。计数器的值为0意味着所有共享资源都被其他线程占用了，所以当前想要使用资源的线程，必须等待其中一个资源被释放。



Semaphore就是JDK中所提供的一个信号量组件。

Semaphore基本用法



1

实例化Semaphore对象（初始值设置为2，表示有两个共享资源）：

```
Semaphore semaphore = new Semaphore(2);
```

2

线程函数中的代码：

```
// 其它代码  
semaphore.acquire();  
...  
semaphore.release()  
// 其它代码
```

“临界区”



Semaphore的构造函数:

```
public Semaphore(int permits)
```

类Semaphore的构造函数中的permits参数是“许可”的意思，代表同一时间内，最多允许多少个线程同时执行acquire()和release()之间的代码，你可以把它看成是可供线程访问的资源数量。

Semaphore使用示例-WhatIsSemaphore



示例创建的Semaphore包容两个运行许可证。这也就是说，一次可以有二个线程投入运行：

```
public class WhatIsSemaphore {  
  
    //有两个运行许可证  
    static Semaphore semaphore = new Semaphore(permits: 2);  
    //用于生成随机的休眠时间  
    static Random random = new Random();  
}
```

线程函数



```
Runnable threadFunc = () -> {
    String threadName = Thread.currentThread().getName();
    try {
        System.out.printf("%s申请获取运行许可证,当前可用数目: %d\n", threadName,
            semaphore.availablePermits());
        semaphore.acquire();
        int sleepTime = random.nextInt( bound: 10) + 1;
        System.out.printf("%s获得了运行许可证, 运行需时%d秒,当前剩余许可证数: %d\n",
            threadName,
            sleepTime,
            semaphore.availablePermits());
        TimeUnit.SECONDS.sleep(sleepTime);
        semaphore.release();
        System.out.printf("\n%s结束运行, 归还运行许可证。 \n\n", threadName);
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
};
```

使用acquire()方法申请许可证, release()方法归还许可证, 这两个方法必须配对。

创建线程并运行.....



```
System.out.println("运行许可证数目: 2, 启动线程: 5");
Thread[] threads = new Thread[5];
for (int i = 1; i <= threads.length; i++) {
    var thread = new Thread(threadFunc);
    thread.setName("工作线程-" + i);
    threads[i - 1] = thread;
    thread.start();
}
//等待所有线程运行结束
Arrays.stream(threads).forEach(th -> {
    try {
        th.join();
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
});
System.out.println("所有线程均已结束。");
```

示例创建了5个线程，投入运行，之后，使用join方法等待所有线程运行结束.....

示例截图



```
Run: WhatIsSemaphore x
"C:\Program Files\Java\jdk-17\bin\java.exe" --modu
运行许可证数目: 2, 启动线程: 5
工作线程-1申请获取运行许可证,当前可用数目: 2
工作线程-5申请获取运行许可证,当前可用数目: 2
工作线程-4申请获取运行许可证,当前可用数目: 2
工作线程-3申请获取运行许可证,当前可用数目: 2
工作线程-2申请获取运行许可证,当前可用数目: 2
工作线程-5获得了运行许可证,运行需时5秒,当前剩余许可证数: 0
工作线程-1获得了运行许可证,运行需时5秒,当前剩余许可证数: 1

工作线程-1结束运行,归还运行许可证。

工作线程-2获得了运行许可证,运行需时10秒,当前剩余许可证数: 0
工作线程-4获得了运行许可证,运行需时6秒,当前剩余许可证数: 1

工作线程-5结束运行,归还运行许可证。
```

五个线程同时申请，每轮只有两个线程可以得到许可证运行，其余的阻塞等待。

每个线程运行的时间不一样，先运行结束的线程，归还许可证之后，另一个原先被阻塞的线程就可以从信号量对象那里得到许可证，然后它就可以运行了。

申请与释放多个许可



一个线程，一次可以申请多个许可：

```
semaphore.acquire(n);
```

也可以一次返回多个许可：

```
semaphore.release(n);
```

“包场”

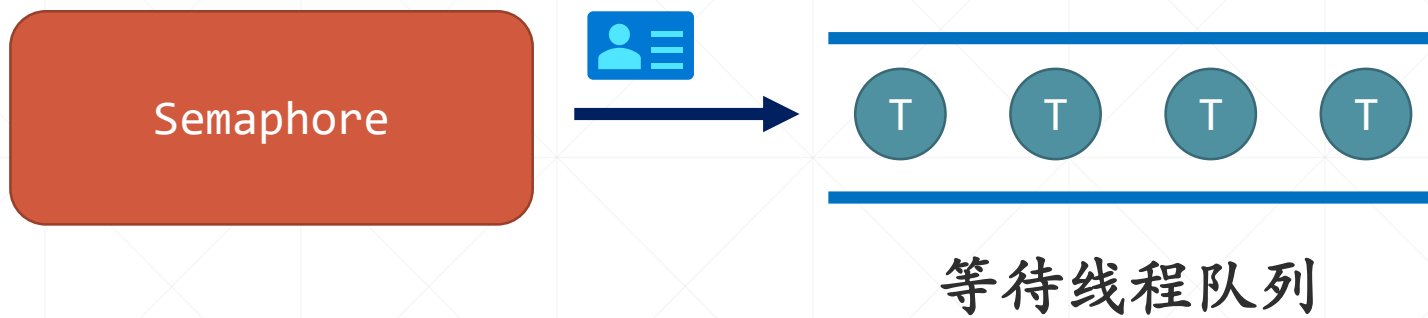
使用`drainPermits()`可以让线程获取所有剩余的许可证。



```
public class DrainPermits {  
    public static void main(String[] args) throws InterruptedException {  
        var semaphore = new Semaphore(permits: 10);  
        semaphore.acquire();  
        semaphore.acquire();  
        System.out.println("剩余许可证: " + semaphore.availablePermits());  
        int result = semaphore.drainPermits();  
        System.out.printf("一口气申请了%d个许可证, 剩余%d个许可证。 %n",  
            result, semaphore.availablePermits());  
    }  
}
```

Run: DrainPermits x

```
"C:\Program Files\Java\jdk-17\bin\java.exe"  
剩余许可证: 8  
一口气申请了8个许可证, 剩余0个许可证。  
  
Process finished with exit code 0
```



可以调用`getQueueLength`方法获取等待队列的长度

一次只允许一个
线程运行

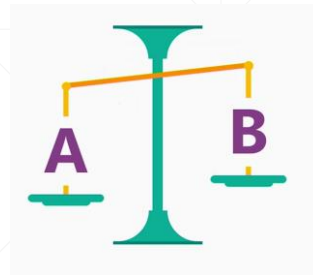


```
public class ThreadQueue {  
    static Semaphore semaphore = new Semaphore(permits: 1);  
    public static void main(String[] args) {  
        Runnable threadFunc = () -> {  
            try {  
                semaphore.acquire();  
                System.out.printf("当前有%d个线程在等待.....\n", semaphore.getQueueLength());  
                Thread.sleep(millis: 1000);  
                System.out.println(Thread.currentThread().getName() + "退出。");  
            } catch (InterruptedException e) {  
                e.printStackTrace();  
            } finally {  
                semaphore.release();  
            }  
        };  
        Thread[] threads = new Thread[5];  
        for (int i = 0; i < threads.length; i++) {  
            threads[i] = new Thread(threadFunc);  
            threads[i].start();  
        }  
    }  
}
```

启动五个线程运行

```
Run: ThreadQueue x  
"C:\Program Files\Java\jdk-17\bin\java.exe"  
当前有0个线程在等待.....  
Thread-1退出。  
当前有3个线程在等待.....  
Thread-2退出。  
当前有2个线程在等待.....  
Thread-4退出。  
当前有1个线程在等待.....  
Thread-3退出。  
当前有0个线程在等待.....  
Thread-0退出。  
  
Process finished with exit code 0
```

公平与非公平信号量



构造信号量时，可以设定它采用“公平”还是“不公平”策略：

Creates a Semaphore with the given number of permits and the given fairness setting.

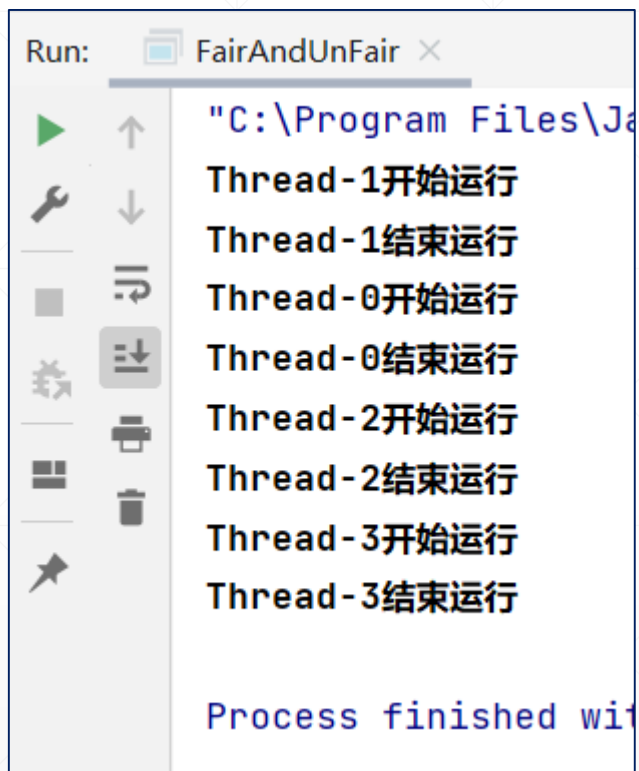
Params: permits – the initial number of permits available. This value may be negative, in which case releases must occur before any acquires will be granted.

fair – true if this semaphore will guarantee first-in first-out granting of permits under contention, else false

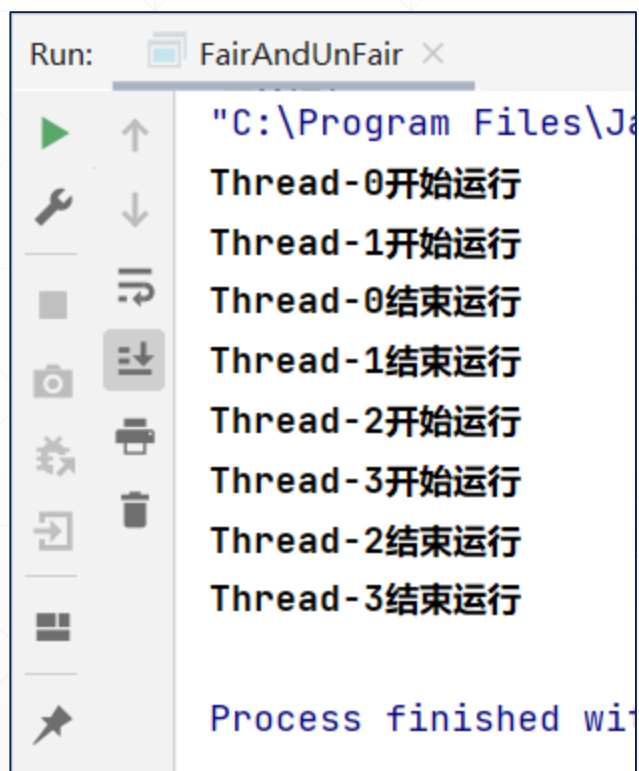
@Contract(pure = true)

```
public Semaphore(int permits, boolean fair) {  
    sync = fair ? new FairSync(permits) : new NonfairSync(permits);  
}
```

运行截图



```
fair = true
```



```
fair = false
```

需要多次运行，才可以看出一点规律。

避免线程长久阻塞导致死锁



有参方法tryAcquire()的作用是尝试地获得许可，如果获取不到则立即返回false，线程不会阻塞等待：

```
public boolean tryAcquire()
```

```
public boolean tryAcquire(int permits)
```

```
public boolean tryAcquire(long timeout, TimeUnit unit)
```


tryAcquire()示例



```
public class AvoidBlock {
    static Semaphore semaphore = new Semaphore(permits: 1);
    public static void main(String[] args) {
        Runnable threadFunc = () -> {
            var threadName : String = Thread.currentThread().getName();
            if (semaphore.tryAcquire()) {
                System.out.println(threadName + "获取了运行许可。");
                try {
                    for(int i=0;i<5;i++){
                        System.out.print(threadName+"["+i+"]");
                        TimeUnit.SECONDS.sleep( timeout: 1);
                    }
                    System.out.println();
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
                semaphore.release();
            } else {
                System.out.println(threadName + "未获得运行许可。");
            }
            System.out.println(threadName+"退出");
        };
        for (int i = 0; i < 5; i++) {
            new Thread(threadFunc).start();
        }
    }
}
```

示例启动五个线程，只有第一个线程得到了许可，其它的只检查了一次，得不到许可就立即退出。

```
Run: AvoidBlock x
"C:\Program Files\Java\jdk-17\bin\java.exe" --module-path "C:
Thread-0获取了运行许可。
Thread-3未获得运行许可。
Thread-4未获得运行许可。
Thread-3退出
Thread-4退出
Thread-2未获得运行许可。
Thread-2退出
Thread-1未获得运行许可。
Thread-1退出
Thread-0[0],Thread-0[1],Thread-0[2],Thread-0[3],Thread-0[4],
Thread-0退出

Process finished with exit code 0
```